

THE UNIVERSITY OF AZAD JAMMU & KASHMIR
MUZAFFARABAD

DATABASE MANAGEMENT SYSTEM LAB

Lab 13 — Open-Ended Lab Report
Car Rental Management System

Name	Tayyab Hanif
Roll No	2024-SE-11
Semester	04
Course Title	Database Management System
Submitted To	Engr. Awais Rathore
Lab No.	13 (Open-Ended Lab)

1. Introduction & Problem Scenario

CarGo Rentals is a local car rental company that rents vehicles to customers on a daily or weekly basis. The company currently manages customer, vehicle, rental, return, and payment records using spreadsheets. This manual approach has led to duplicate information, inconsistent records, and difficulty in generating meaningful reports as the number of rentals has grown.

This lab designs and implements a relational database solution for CarGo Rentals. The system stores customer and vehicle information, records rentals and returns, calculates rental charges automatically, and maintains payment details. A core business rule enforced by the system is that a vehicle must not be available for a new rental while it is already rented. The database also supports management reporting needs such as customer rental history, currently rented vehicles, and vehicle rental activity.

1.1 Objectives

- Design a normalized relational database for a real-world car rental scenario.
- Apply primary keys, foreign keys, and appropriate constraints (NOT NULL, UNIQUE, CHECK, DEFAULT).
- Demonstrate the normalization process from an unnormalized structure to 3NF.
- Write JOIN queries to answer specific business questions.
- Implement a VIEW, a TRIGGER, and a STORED PROCEDURE that enforce and support business logic.
- Identify a design/query inefficiency and propose a justified optimization.

1.2 Tools Used

- XAMPP (Apache + MySQL)
- phpMyAdmin — for database creation, execution, and screenshots
- MySQL / SQL (DDL, DML, Views, Triggers, Stored Procedures)

2. Task 1 — Database Design & Implementation

Based on the problem scenario, five tables are required: customers, vehicles, rentals, returns, and payments. Rentals, returns, and payments are kept as separate tables (rather than one wide table) because they represent distinct events in time — a rental may exist without a return or a payment yet, and combining them would reintroduce the redundancy problems addressed in Task 2 (Normalization).

2.1 Table: customers

Column	Type	Constraint / Key	Description
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each customer
full_name	VARCHAR(100)	NOT NULL	Customer's full name
phone	VARCHAR(15)	NOT NULL, UNIQUE	Contact number; prevents duplicate customer records
cnic	VARCHAR(15)	UNIQUE	National ID for identity verification
address	VARCHAR(150)	—	Residential address

2.2 Table: vehicles

Column	Type	Constraint / Key	Description
vehicle_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each vehicle
vehicle_number	VARCHAR(20)	NOT NULL, UNIQUE	Registration/number plate
model	VARCHAR(50)	NOT NULL	Vehicle make and model
daily_rate	DECIMAL(10,2)	NOT NULL, CHECK (daily_rate > 0)	Rental cost per day
status	VARCHAR(15)	NOT NULL, DEFAULT 'Available', CHECK (status IN ('Available', 'Rented'))	Tracks current availability; maintained by trigger

2.3 Table: rentals

Column	Type	Constraint / Key	Description
rental_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each rental transaction
customer_id	INT	NOT NULL, FOREIGN KEY → customers	Customer who rented the vehicle
vehicle_id	INT	NOT NULL, FOREIGN KEY → vehicles	Vehicle being rented
rental_date	DATE	NOT NULL	Date the rental begins
expected_return_date	DATE	NOT NULL, CHECK (expected_return_date > rental_date)	Agreed return date

2.4 Table: returns

Column	Type	Constraint / Key	Description
return_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each return record
rental_id	INT	NOT NULL, UNIQUE, FOREIGN KEY → rentals	Rental being closed (one return per rental)
actual_return_date	DATE	NOT NULL	Date the vehicle was physically returned
total_days	INT	NOT NULL, CHECK (total_days > 0)	Billable duration of the rental

2.5 Table: payments

Column	Type	Constraint / Key	Description
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique identifier for each payment
rental_id	INT	NOT NULL, FOREIGN KEY → rentals	Rental being paid for
amount	DECIMAL(10,2)	NOT NULL, CHECK (amount >= 0)	Amount paid
payment_date	DATE	NOT NULL, DEFAULT (CURRENT_DATE)	Date payment was received
payment_method	VARCHAR(20)	DEFAULT 'Cash'	Cash, Card, or Online Transfer

2.6 Relationships / ER Summary

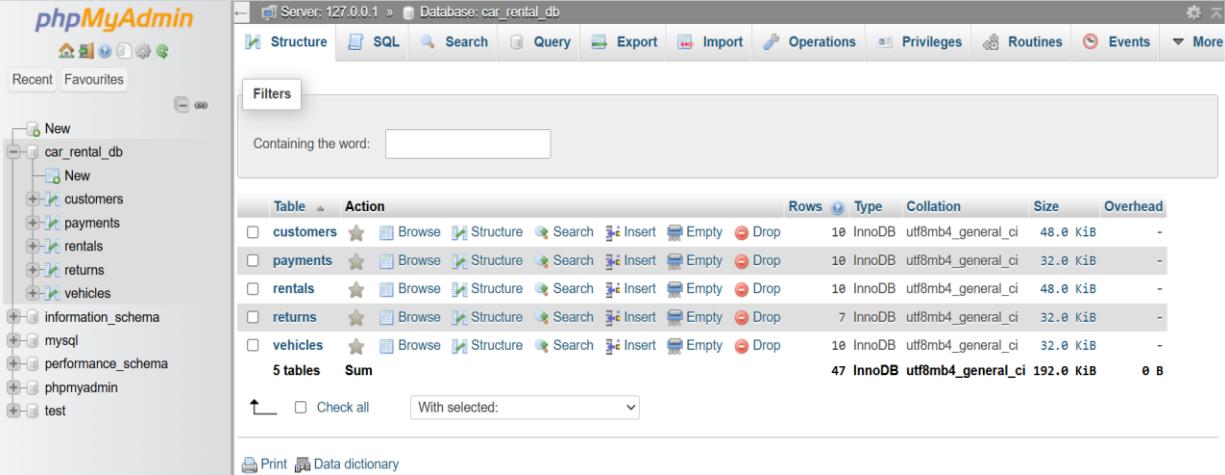
- customers → rentals : one customer can have many rentals (1:N)
- vehicles → rentals : one vehicle can appear in many rentals over time (1:N)
- rentals → returns : one rental has at most one return (1:1)
- rentals → payments : one rental can have one or more payments (1:N)

2.7 Design Justification

Separating returns and payments from rentals avoids storing NULL-heavy rows for rentals that are still active, and allows multiple/partial payments per rental without duplicating rental or vehicle data. The status column on vehicles gives an O(1) way to check availability instead of scanning historical rentals, and is kept consistent automatically by the trigger described in Task 5.

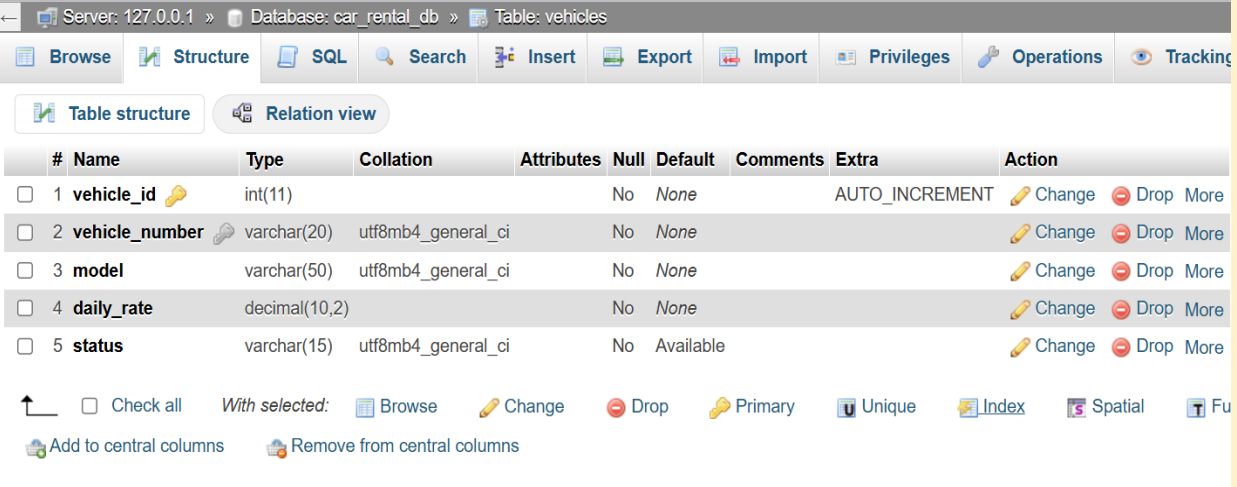
2.8 Implementation in phpMyAdmin

The database and all five tables were created and populated using the SQL file submitted alongside this report, executed through the SQL tab in phpMyAdmin.



The screenshot shows the phpMyAdmin interface for the 'car_rental_db' database. The left sidebar lists the database and its tables: customers, payments, rentals, returns, and vehicles. The main panel displays the 'Structure' tab for the database, showing a list of tables with their respective actions (Browse, Structure, Search, Insert, Empty, Drop). The tables are: customers (10 rows), payments (10 rows), rentals (10 rows), returns (7 rows), and vehicles (10 rows). A summary row shows 5 tables with a total of 47 rows.

Table	Action	Rows	Type	Collation	Size	Overhead
customers	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	48.0 KiB	-
payments	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	32.0 KiB	-
rentals	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	48.0 KiB	-
returns	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_general_ci	32.0 KiB	-
vehicles	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_general_ci	32.0 KiB	-
5 tables	Sum	47	InnoDB	utf8mb4_general_ci	192.0 KiB	0 B



The screenshot shows the 'Table structure' tab for the 'vehicles' table in the 'car_rental_db' database. The table has five columns: vehicle_id (int(11), primary key, AUTO_INCREMENT), vehicle_number (varchar(20)), model (varchar(50)), daily_rate (decimal(10,2)), and status (varchar(15)). The interface includes options to browse, change, drop, or add primary/unique/index/spatial columns.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	vehicle_id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	vehicle_number	varchar(20)	utf8mb4_general_ci		No	None			Change Drop More
3	model	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
4	daily_rate	decimal(10,2)			No	None			Change Drop More
5	status	varchar(15)	utf8mb4_general_ci		No	Available			Change Drop More

Server: 127.0.0.1 » Database: car_rental_db » Table: customers

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	customer_id	full_name	phone	cnic	address
☐ Edit Copy Delete	1	Ali Raza	0300-1234567	35202-1111111-1	Muzaffarabad
☐ Edit Copy Delete	2	Bilal Ahmed	0301-2345678	35202-2222222-2	Rawalakot
☐ Edit Copy Delete	3	Sana Tariq	0302-3456789	35202-3333333-3	Mirpur
☐ Edit Copy Delete	4	Hina Malik	0303-4567890	35202-4444444-4	Bagh
☐ Edit Copy Delete	5	Usman Ghani	0304-5678901	35202-5555555-5	Muzaffarabad
☐ Edit Copy Delete	6	Ayesha Noor	0305-6789012	35202-6666666-6	Kotli
☐ Edit Copy Delete	7	Zainab Fatima	0306-7890123	35202-7777777-7	Neelum
☐ Edit Copy Delete	8	Omar Sheikh	0307-8901234	35202-8888888-8	Hattian Bala
☐ Edit Copy Delete	9	Farhan Iqbal	0308-9012345	35202-9999999-9	Muzaffarabad
☐ Edit Copy Delete	10	Mahnoor Aslam	0309-0123456	35202-1010101-0	Bhimber

☐ Check all | With selected: Edit Copy Delete Export

Server: 127.0.0.1 » Database: car_rental_db » Table: vehicles

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	vehicle_id	vehicle_number	model	daily_rate	status
☐ Edit Copy Delete	1	ABC-123	Toyota Corolla	5000.00	Available
☐ Edit Copy Delete	2	XYZ-456	Honda Civic	5500.00	Rented
☐ Edit Copy Delete	3	LEA-789	Suzuki Cultus	3000.00	Available
☐ Edit Copy Delete	4	GJ-321	Toyota Hiace	8000.00	Available
☐ Edit Copy Delete	5	BC-654	Honda City	4500.00	Rented
☐ Edit Copy Delete	6	KL-987	Suzuki Alto	2500.00	Available
☐ Edit Copy Delete	7	MN-159	Toyota Fortuner	12000.00	Available
☐ Edit Copy Delete	8	OP-753	Honda BR-V	6000.00	Rented
☐ Edit Copy Delete	9	QR-246	Suzuki Wagon R	2800.00	Available
☐ Edit Copy Delete	10	ST-864	Toyota Yaris	4200.00	Available

☐ Check all | With selected: Edit Copy Delete Export

3. Task 2 — Normalization (1NF → 3NF)

The company's original record structure stores all rental information in a single flat table:

RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	DailyRate
RentalDate	ReturnDate	PaymentAmount			
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000
2026-09-01	2026-09-04	15000			

This structure mixes facts about the customer, the vehicle, and the rental transaction into one row, which causes update, insertion, and deletion anomalies. For example, if Ali Khan's phone number changes, every rental row for him must be updated; if a vehicle's daily rate changes, historical rows may become inconsistent; and a new vehicle cannot be added until it is rented.

3.1 First Normal Form (1NF)

A relation is in 1NF if all attributes contain atomic (indivisible) values and there are no repeating groups. In the given table, every cell already holds a single value (one customer, one vehicle, one date per row), so the table satisfies 1NF. It still mixes independent entities in a single row, which 2NF and 3NF resolve below.

3.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and every non-key attribute is fully functionally dependent on the whole primary key. The primary key here is RentalID, but CustomerName and CustomerPhone depend only on the customer, and VehicleModel and DailyRate depend only on the vehicle — a partial dependency on part of what should be a composite real-world key. Decomposing removes this:

- customers (customer_id, full_name, phone)
- vehicles (vehicle_id, vehicle_number, model, daily_rate)
- rentals (rental_id, customer_id, vehicle_id, rental_date, return_date, payment_amount)

This removes the redundancy of storing a customer's phone number on every rental row, and allows a vehicle's rate to be stored once instead of once per rental record.

3.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and has no transitive dependencies, i.e. no non-key attribute depends on another non-key attribute rather than directly on the key. In the 2NF rentals table, ReturnDate and PaymentAmount are transitively tied together even though a rental can exist, be returned, and be paid for at three different points in time. Splitting these into independent tables — each keyed by rental_id — removes the transitive dependency and produces the final 3NF design implemented in Task 1:

- customers (customer_id PK, full_name, phone, cnic, address)
- vehicles (vehicle_id PK, vehicle_number, model, daily_rate, status)
- rentals (rental_id PK, customer_id FK, vehicle_id FK, rental_date, expected_return_date)

- returns (return_id PK, rental_id FK, actual_return_date, total_days)
- payments (payment_id PK, rental_id FK, amount, payment_date, payment_method)

3.4 Summary of Anomalies Resolved

Anomaly Type	Present In Flat Table	Resolved At
Update anomaly	Changing a customer's phone or a vehicle's rate requires updating many rows	2NF
Insertion anomaly	A new vehicle cannot be added until it is rented	2NF
Deletion anomaly	Deleting the only rental for a customer/vehicle erases their details entirely	2NF
Transitive redundancy	Payment and return status are tied together even though they occur at different times	3NF

4. Task 3 — JOIN Queries

The following queries answer the business questions listed in the lab manual, using appropriate JOIN types so that customers or vehicles with no matching rentals are still included where required.

Query 1: Customer name, vehicle number, model, rental date and return date for every rental

```
SELECT c.full_name, v.vehicle_number, v.model,
       r.rental_date, ret.actual_return_date
FROM rentals r
JOIN customers c ON r.customer_id = c.customer_id
JOIN vehicles v  ON r.vehicle_id  = v.vehicle_id
LEFT JOIN returns ret ON ret.rental_id = r.rental_id;
```

An INNER JOIN is used for customers and vehicles because every rental must reference a valid customer and vehicle. A LEFT JOIN is used for returns so that rentals which have not yet been returned still appear, with a NULL return date.

✓ Showing rows 0 - 9 (10 total, Query took 0.0012 seconds.)

```
SELECT c.full_name, v.vehicle_number, v.model, r.rental_date, ret.actual_return_date FROM rentals r JOIN customers c ON r.customer_id = c.customer_id JOIN vehicles v ON r.vehicle_id = v.vehicle_id LEFT JOIN returns ret ON ret.rental_id = r.rental_id;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

full_name	vehicle_number	model	rental_date	actual_return_date
Ali Raza	ABC-123	Toyota Corolla	2026-09-01	2026-09-04
Bilal Ahmed	XYZ-456	Honda Civic	2026-09-18	NULL
Sana Tariq	LEA-789	Suzuki Cultus	2026-09-02	2026-09-05
Hina Malik	GJ-321	Toyota Hiace	2026-09-03	2026-09-06
Usman Ghani	BC-654	Honda City	2026-09-19	NULL
Ayesha Noor	KL-987	Suzuki Alto	2026-09-04	2026-09-07
Zainab Fatima	MN-159	Toyota Fortuner	2026-09-05	2026-09-10
Omar Sheikh	OP-753	Honda BR-V	2026-09-17	NULL
Farhan Iqbal	QR-246	Suzuki Wagon R	2026-09-06	2026-09-08
Mahnoor Aslam	ST-864	Toyota Yaris	2026-09-07	2026-09-09

☒ Console

Query 2: All customers and the vehicles they have rented, including customers who never rented

```
SELECT c.full_name, v.vehicle_number, v.model
FROM customers c
LEFT JOIN rentals r ON c.customer_id = r.customer_id
LEFT JOIN vehicles v ON r.vehicle_id = v.vehicle_id
ORDER BY c.full_name;
```

Starting the LEFT JOIN chain from customers ensures every customer row is preserved even when there is no matching rental.

✓ Showing rows 0 - 11 (12 total, Query took 0.0071 seconds.) [full_name: ALI RAZA... - ZAINAB FATIMA...]

```
SELECT c.full_name, v.vehicle_number, v.model FROM customers c LEFT JOIN rentals r ON c.customer_id = r.customer_id LEFT JOIN vehicles v ON r.vehicle_id = v.vehicle_id ORDER BY c.full_name;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

full_name	vehicle_number	model
Ali Raza	ABC-123	Toyota Corolla
Ayesha Noor	KL-987	Suzuki Alto
Bilal Ahmed	XYZ-456	Honda Civic
Farhan Iqbal	QR-246	Suzuki Wagon R
Hina Malik	GJ-321	Toyota Hiace
Kamran Yousaf	NULL	NULL
Mahnoor Aslam	ST-864	Toyota Yaris
Omar Sheikh	OP-753	Honda BR-V
Rabia Sultana	NULL	NULL
Sana Tariq	LEA-789	Suzuki Cultus
Usman Ghani	BC-654	Honda City

☒ Console

Query 3: All vehicles and their current rental information, including vehicles not currently rented

```
SELECT v.vehicle_number, v.model, v.status,
       c.full_name AS current_customer,
       r.rental_date, r.expected_return_date
FROM vehicles v
LEFT JOIN rentals r ON v.vehicle_id = r.vehicle_id
                   AND v.status = 'Rented'
LEFT JOIN customers c ON r.customer_id = c.customer_id
ORDER BY v.vehicle_number;
```

The join condition is restricted to `v.status = 'Rented'` so that only the active rental (if any) is attached to each vehicle; vehicles that are Available appear with NULL rental details.

Showing rows 0 - 9 (10 total, Query took 0.0008 seconds.) [vehicle_number: ABC-123... - XYZ-456...]

```
SELECT v.vehicle_number, v.model, v.status, c.full_name AS current_customer, r.rental_date, r.expected_return_date FROM vehicle
JOIN rentals r ON v.vehicle_id = r.vehicle_id AND v.status = 'Rented' LEFT JOIN customers c ON r.customer_id = c.customer_id OR
v.vehicle_number;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

vehicle_number	model	status	current_customer	rental_date	expected_return_date
ABC-123	Toyota Corolla	Available	NULL	NULL	NULL
BC-654	Honda City	Rented	Usman Ghani	2026-09-19	2026-09-22
GJ-321	Toyota Hiace	Available	NULL	NULL	NULL
KL-987	Suzuki Alto	Available	NULL	NULL	NULL
LEA-789	Suzuki Cultus	Available	NULL	NULL	NULL
MN-159	Toyota Fortuner	Available	NULL	NULL	NULL
OP-753	Honda BR-V	Rented	Omar Sheikh	2026-09-17	2026-09-24
QR-246	Suzuki Wagon R	Available	NULL	NULL	NULL
ST-864	Toyota Yaris	Available	NULL	NULL	NULL
XY7-456	Honda Civic	Rented	Bilal Ahmed	2026-09-18	2026-09-25

☒ Console

Query 4: Total number of rentals made by each customer, including customers with zero rentals

```
SELECT c.full_name, COUNT(r.rental_id) AS total_rentals
FROM customers c
LEFT JOIN rentals r ON c.customer_id = r.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_rentals DESC;
```

`COUNT(r.rental_id)` is used instead of `COUNT(*)` so that customers with no matching rental (a NULL row from the LEFT JOIN) are correctly counted as 0, not 1.

✓ Showing rows 0 - 11 (12 total, Query took 0.0058 seconds.)

```
SELECT c.full_name, COUNT(r.rental_id) AS total_rentals FROM customers c LEFT JOIN rentals r ON c.customer_id = r.customer_id GROUP BY c.customer_id, c.full_name ORDER BY total_rentals DESC;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

full_name	total_rentals
Bilal Ahmed	1
Mahnoor Aslam	1
Zainab Fatima	1
Hina Malik	1
Ali Raza	1
Farhan Iqbal	1
Ayesha Noor	1
Sana Tariq	1
Omar Sheikh	1
Usman Ghani	1
Rabia Sultana	0
Console saf	0

5. Task 4 — View

A single consolidated view is created to give management a one-stop rental report combining customer, vehicle, rental, and payment information without needing to repeat the underlying joins every time.

```
CREATE VIEW rental_summary AS
SELECT r.rental_id,
       c.full_name      AS customer_name,
       c.phone          AS customer_phone,
       v.vehicle_number,
       v.model          AS vehicle_model,
       v.daily_rate,
       r.rental_date,
       r.expected_return_date,
       ret.actual_return_date,
       p.amount         AS amount_paid,
       p.payment_date
FROM rentals r
JOIN customers c      ON r.customer_id = c.customer_id
JOIN vehicles v       ON r.vehicle_id  = v.vehicle_id
LEFT JOIN returns ret  ON ret.rental_id = r.rental_id
LEFT JOIN payments p   ON p.rental_id  = r.rental_id;
```

Example usage:

```
SELECT * FROM rental_summary ORDER BY rental_date DESC;
```

5.1 Why This View Is Useful

- It hides the complexity of a five-table join behind one simple, reusable query for staff who are not familiar with SQL joins.
- It gives a single, consistent source for the rental report requested by management, reducing the risk of two staff members writing slightly different (and inconsistent) queries.
- It can be reused directly for reporting tools, exports, or as the basis for further filtering (e.g. WHERE rental_date >= ...) without duplicating join logic.

Server: 127.0.0.1 » Database: car_rental_db

StructureSQLSearchQueryExportImportOperationsPrivilegesRoutinesEventsMo

Show query box

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
CREATE VIEW rental_summary AS SELECT r.rental_id, c.full_name AS customer_name, c.phone AS customer_phone, v.vehicle_number, v.model AS vehicle_model, v.daily_rate, r.rental_date, r.expected_return_date, ret.actual_return_date, p.amount AS amount_paid, p.payment_date FROM rentals r JOIN customers c ON r.customer_id = c.customer_id JOIN vehicles v ON r.vehicle_id = v.vehicle_id LEFT JOIN returns ret ON ret.rental_id = r.rental_id LEFT JOIN payments p ON p.rental_id = r.rental_id;
```

[Edit inline][Edit][Create PHP code]

Showing rows 0 - 9 (10 total, Query took 0.0066 seconds.) [rental_date: 2026-09-19... - 2026-09-01...]

```
SELECT * FROM rental_summary ORDER BY rental_date DESC;
```

☐ Profiling [Edit inline][Edit][Explain SQL][Create PHP code][Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	rental_id	customer_name	customer_phone	vehicle_number	vehicle_model	daily_rate	rental_date	expected
☐ Edit Copy Delete	5	Usman Ghani	0304-5678901	BC-654	Honda City	4500.00	2026-09-19	2026-09-2
☐ Edit Copy Delete	2	Bilal Ahmed	0301-2345678	XYZ-456	Honda Civic	5500.00	2026-09-18	2026-09-2
☐ Edit Copy Delete	8	Omar Sheikh	0307-8901234	OP-753	Honda BR-V	6000.00	2026-09-17	2026-09-2
☐ Edit Copy Delete	10	Mahnoor Aslam	0309-0123456	ST-864	Toyota Yaris	4200.00	2026-09-07	2026-09-1
☐ Edit Copy Delete	9	Farhan Iqbal	0308-9012345	QR-246	Suzuki Wagon R	2800.00	2026-09-06	2026-09-0
☐ Edit Copy Delete	7	Zainab Fatima	0306-7890123	MN-159	Toyota Fortuner	12000.00	2026-09-05	2026-09-1
☐ Edit Copy Delete	6	Ayesha Noor	0305-6789012	KL-987	Suzuki Alto	2500.00	2026-09-04	2026-09-0
☐ Edit Copy Delete	4	Hina Malik	0303-4567890	GJ-321	Toyota Hiace	8000.00	2026-09-03	2026-09-0
☒ Console Edit Copy Delete	3	Sana Tariq	0302-3456789	LEA-789	Suzuki Cultus	3000.00	2026-09-02	2026-09-0

6. Task 5 — Trigger

The lab manual's submission requirements and marking rubric both require a TRIGGER, built around the scenario's core business rule: a vehicle must not be available for another rental while it is already rented. Two complementary triggers implement this rule end-to-end — one that blocks a new rental on an already-rented vehicle, and one that frees the vehicle back up when it is returned.

6.1 Trigger 1 — Prevent Double Booking

```
DELIMITER $$
CREATE TRIGGER trg_before_rental_insert
BEFORE INSERT ON rentals
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(15);

    SELECT status INTO v_status
    FROM vehicles
    WHERE vehicle_id = NEW.vehicle_id;

    IF v_status = 'Rented' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'This vehicle is already rented and unavailable.';
    ELSE
        UPDATE vehicles
        SET status = 'Rented'
        WHERE vehicle_id = NEW.vehicle_id;
    END IF;
END$$
DELIMITER ;
```

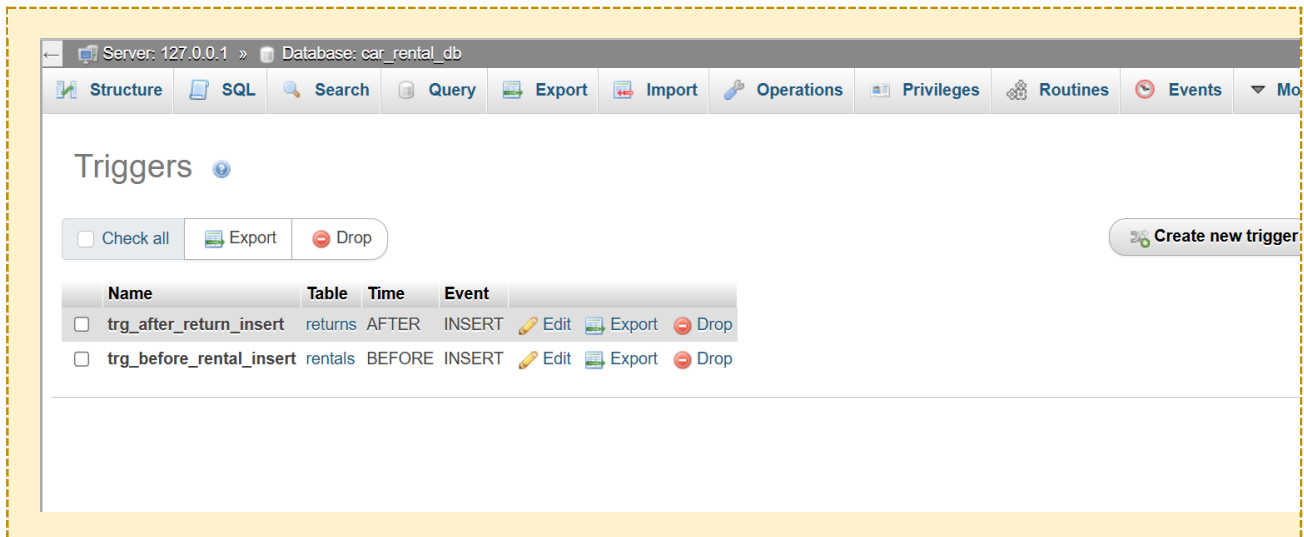
6.2 Trigger 2 — Release Vehicle on Return

```
DELIMITER $$
CREATE TRIGGER trg_after_return_insert
AFTER INSERT ON returns
FOR EACH ROW
BEGIN
    UPDATE vehicles v
    JOIN rentals r ON r.vehicle_id = v.vehicle_id
    SET v.status = 'Available'
    WHERE r.rental_id = NEW.rental_id;
END$$
DELIMITER ;
```

6.3 Testing the Trigger

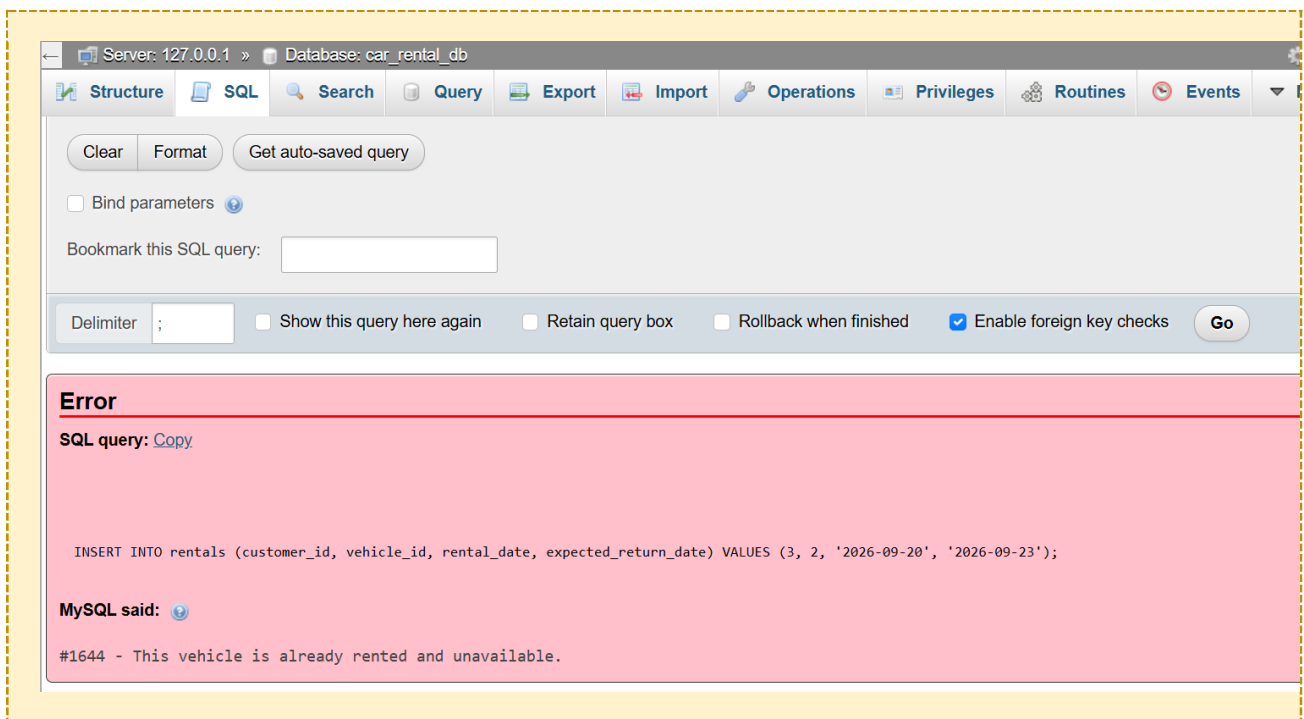
To demonstrate the trigger, attempt to insert a rental for a vehicle whose status is already 'Rented'. The insert should fail with the custom error message, proving the business rule is enforced at the database level rather than relying on application code.

```
-- This should be rejected if vehicle_id = 1 is already Rented:  
INSERT INTO rentals (customer_id, vehicle_id, rental_date, expected_return_date)  
VALUES (2, 1, '2026-09-15', '2026-09-18');
```



The screenshot shows the MySQL Workbench interface with the 'Triggers' tab selected for the 'car_rental_db' database. The interface includes a toolbar with buttons for 'Check all', 'Export', and 'Drop'. A 'Create new trigger' button is also present. Below these buttons is a table listing existing triggers:

Name	Table	Time	Event	
☐ trg_after_return_insert	returns	AFTER	INSERT	Edit Export Drop
☐ trg_before_rental_insert	rentals	BEFORE	INSERT	Edit Export Drop



The screenshot shows the MySQL Workbench interface with the 'SQL' tab selected. The toolbar includes buttons for 'Clear', 'Format', and 'Get auto-saved query'. Below the toolbar, there are options for 'Bind parameters', 'Bookmark this SQL query', and a 'Delimiter' dropdown set to ';'. The 'Go' button is visible. The main area displays an error message:

Error

SQL query: [Copy](#)

```
INSERT INTO rentals (customer_id, vehicle_id, rental_date, expected_return_date) VALUES (3, 2, '2026-09-20', '2026-09-23');
```

MySQL said:

#1644 - This vehicle is already rented and unavailable.

Server: 127.0.0.1 » Database: car_rental_db

Structure SQL Search Query Export Import Operations Privileges Routines Events More

Show query box

✓ 1 row inserted.
Inserted row id: 11 (Query took 0.0003 seconds.)

```
INSERT INTO rentals (customer_id, vehicle_id, rental_date, expected_return_date) VALUES (11, 1, '2026-09-20', '2026-09-23');
```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
-- this will be rental_id 11;
```

[Edit inline] [Edit] [Create PHP code]

Server: 127.0.0.1 » Database: car_rental_db » Table: vehicles

Browse Structure SQL Search Insert Export Import Privileges Operations Tracking More

Show query box

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT vehicle_id, status FROM vehicles WHERE vehicle_id = 1;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Extra options

	vehicle_id	status
☐ Edit ☐ Copy ☐ Delete	1	Rented

↑ ☐ Check all With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table

Structure
SQL
Search
Query
Export
Import
Operations
Privileges
Routines
Events
More

Clear
Format
Get auto-saved query

☐ Bind parameters

Bookmark this SQL query:

Delimiter: ;
☐ Show this query here again
☐ Retain query box
☐ Rollback when finished
☒ Enable foreign key checks
Go

Error

SQL query: [Copy](#)

```
INSERT INTO returns (rental_id, actual_return_date, total_days) VALUES (11, '2026-09-23', 3);
```

MySQL said:

```
#1062 - Duplicate entry '11' for key 'rental_id'
```

Server: 127.0.0.1
Database: car_rental_db
Table: vehicles

Browse
Structure
SQL
Search
Insert
Export
Import
Privileges
Operation

Show query box

Showing rows 0 - 0 (1 total, Query took 0.0002 seconds.)

```
SELECT vehicle_id, status FROM vehicles WHERE vehicle_id = 1;
```

☐ Profiling
[\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all
Number of rows: 25
Filter rows: Search this table

Extra options

	vehicle_id	status
☐	1	Available

☐ Check all
With selected:
[Edit](#)
[Copy](#)
[Delete](#)
[Export](#)

☐ Show all
Number of rows: 25
Filter rows: Search this table

7. Task 6 — Stored Procedure

A stored procedure, RegisterRental, performs the suggested operation: it registers a new rental and calculates the rental charge based on the rental period and the vehicle's daily rate, returning the computed charge through an OUT parameter.

```

DELIMITER $$
CREATE PROCEDURE RegisterRental (
    IN p_customer_id INT,
    IN p_vehicle_id INT,
    IN p_rental_date DATE,
    IN p_expected_return_date DATE,
    OUT p_total_charge DECIMAL(10,2)
)
BEGIN
    DECLARE v_rate DECIMAL(10,2);
    DECLARE v_days INT;

    SELECT daily_rate INTO v_rate
    FROM vehicles
    WHERE vehicle_id = p_vehicle_id;

    SET v_days = DATEDIFF(p_expected_return_date, p_rental_date);
    SET p_total_charge = v_rate * v_days;

    INSERT INTO rentals (customer_id, vehicle_id, rental_date, expected_return_date)
    VALUES (p_customer_id, p_vehicle_id, p_rental_date, p_expected_return_date);
END$$
DELIMITER ;

```

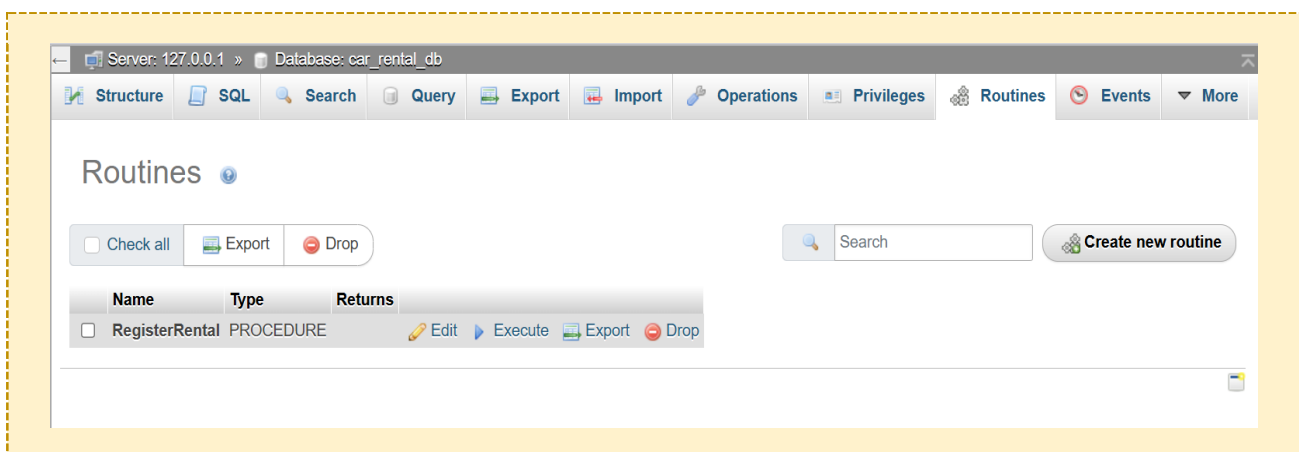
7.1 Calling the Procedure

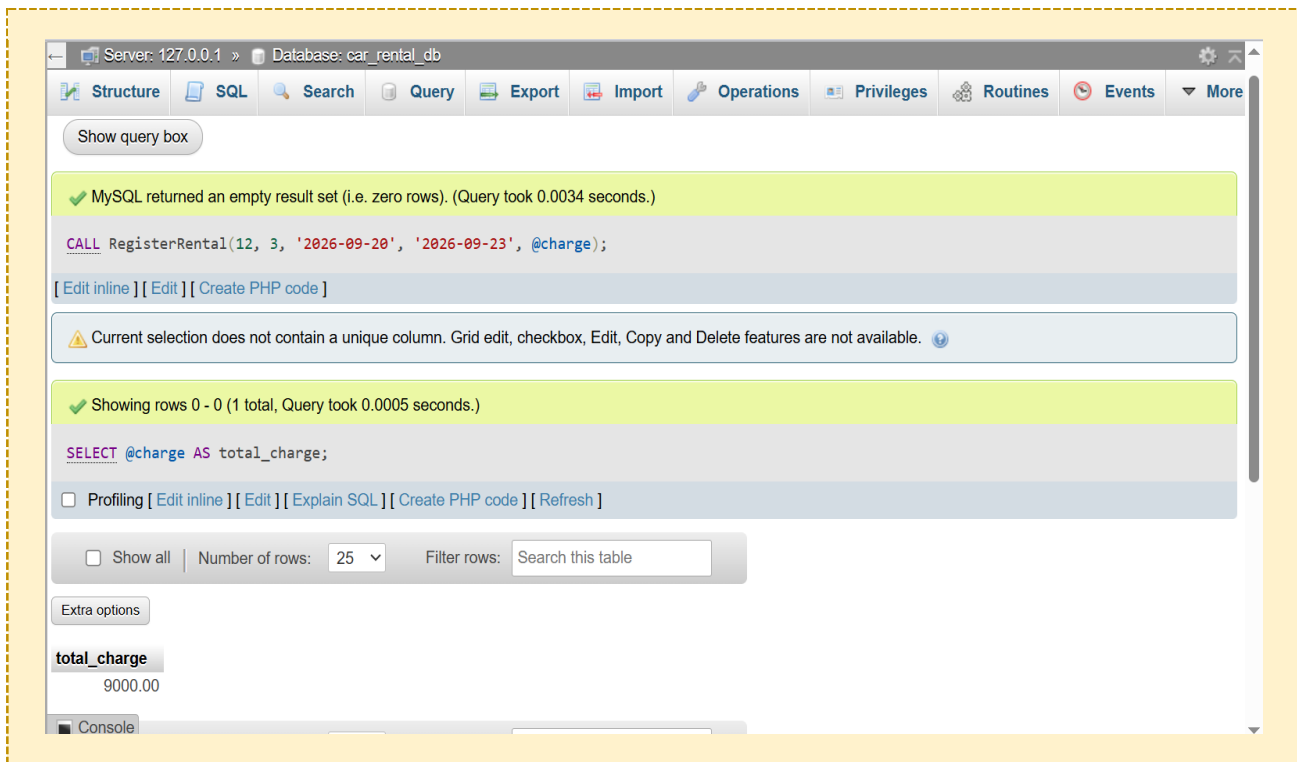
```

CALL RegisterRental(3, 4, '2026-09-20', '2026-09-23', @charge);
SELECT @charge AS total_charge;

```

Inserting into rentals from inside the procedure automatically fires `trg_before_rental_insert` (Task 5), so the double-booking check and the status update happen consistently whether a rental is registered through this procedure or directly via SQL.





8. Task 7 — Optimization Analysis

8.1 Identified Inefficiency

Without the status column and index support on the foreign key columns, checking whether a vehicle is currently available would require scanning the entire rentals table (and joining against returns) every time — for example: "find a rental for this vehicle that has no matching row in returns." As the rentals table grows to thousands of rows, this becomes a full table scan on every availability check, and the same logic could easily be written slightly differently in different queries, risking inconsistent answers about whether a vehicle is free.

8.2 Why This Matters

- Performance: repeated full scans of rentals to determine availability degrade linearly (or worse, with the join to returns) as rental history accumulates.
- Consistency: if availability is computed ad-hoc in multiple queries or in application code instead of one authoritative place, two rentals could be approved for the same vehicle at the same time — a race condition — before either transaction commits.
- Query cost: JOINS on rentals.vehicle_id and rentals.customer_id used throughout Tasks 3–4 rely on the foreign key columns being efficiently searchable; without indexes, these also degrade to full scans.

8.3 Proposed Improvement

Two complementary optimizations are applied. First, the vehicles.status column (maintained automatically by the triggers in Task 5) turns an expensive historical scan into a single indexed lookup on the primary key of

vehicles. Second, explicit indexes are added on the foreign key columns most frequently joined or filtered in Tasks 3 and 4:

```
CREATE INDEX idx_rentals_vehicle ON rentals(vehicle_id);
CREATE INDEX idx_rentals_customer ON rentals(customer_id);
CREATE INDEX idx_payments_rental ON payments(rental_id);
```

With these indexes, MySQL can use an index lookup instead of a full table scan when resolving the JOIN conditions in Task 3's queries and the GROUP BY in Task 3, Query 4, and the status column gives an O(1) availability check that the trigger keeps perfectly in sync with the actual rental/return history.

EXPLAIN on the unoptimized check

Server: 127.0.0.1 > Database: car_rental_db

Structure SQL Search Query Export Import Operations Privileges Routines Events More

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Your SQL query has been executed successfully.

EXPLAIN SELECT * FROM vehicles v WHERE v.vehicle_id = 3 AND NOT EXISTS (SELECT 1 FROM rentals r LEFT JOIN returns ret ON ret.rental_id = r.rental_id WHERE r.vehicle_id = v.vehicle_id AND ret.return_id IS NULL);

[Edit inline] [Edit] [Skip Explain SQL] [Create PHP code]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	v	const	PRIMARY	PRIMARY	4	const	1	
2	DEPENDENT SUBQUERY	r	ref	fk_rentals_vehicle	fk_rentals_vehicle	4	func	1	Using index
2	DEPENDENT SUBQUERY	ret	eq_ref	rental_id	rental_id	4	car_rental_db.r.rental_id	1	Using where; Using index; Not exists

Query results operations

Print Copy to clipboard Create view

Your SQL query has been executed successfully.

EXPLAIN SELECT * FROM vehicles v WHERE NOT EXISTS (SELECT 1 FROM rentals r LEFT JOIN returns ret ON ret.rental_id = r.rental_id WHERE r.vehicle_id = v.vehicle_id AND ret.return_id IS NULL);

[Edit inline] [Edit] [Skip Explain SQL] [Create PHP code]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	v	ALL	NULL	NULL	NULL	NULL	10	Using where
2	MATERIALIZED	r	index	idx_rentals_vehicle	idx_rentals_vehicle	4	NULL	12	Using index
2	MATERIALIZED	ret	eq_ref	rental_id	rental_id	4	car_rental_db.r.rental_id	1	Using where; Using index; Not exists

Query results operations

Print Copy to clipboard Create view

EXPLAIN on the unoptimized check

Server: 127.0.0.1 » Database: car_rental_db

Structure SQL Search Query Export Import Operations Privileges Routines Events More

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

Your SQL query has been executed successfully.

```
EXPLAIN SELECT * FROM vehicles WHERE vehicle_id = 3 AND status = 'Available';
```

[Edit inline] [Edit] [Skip Explain SQL] [Create PHP code]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	NULL	NULL	NULL	NULL	NULL	NULL	NULL	Impossible WHERE noticed after reading const tables

Query results operations

Print Copy to clipboard Create view

Your SQL query has been executed successfully.

```
EXPLAIN SELECT * FROM vehicles WHERE status = 'Available';
```

[Edit inline] [Edit] [Skip Explain SQL] [Create PHP code]

Extra options

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	vehicles	ALL	NULL	NULL	NULL	NULL	10	Using where

Query results operations

Print Copy to clipboard Create view

9. Conclusion

This lab designed and implemented a complete relational database for CarGo Rentals, moving from an unnormalized flat record to a fully normalized (3NF) five-table schema with appropriate primary keys, foreign keys, and constraints. JOIN queries were written to answer all required business questions using correct INNER and LEFT JOIN logic, and a VIEW, two TRIGGERS, and a STORED PROCEDURE were implemented to enforce the core business rule — that a vehicle cannot be double-booked — directly at the database level. Finally, an optimization was identified and justified, showing how a maintained status column combined with foreign-key indexes improves both the performance and the consistency of the system. Together, these components demonstrate a database solution that is not only functionally correct but also efficient and resistant to the anomalies present in the original spreadsheet-based approach.